

Missing Concepts

The important JavaScript, React & TypeScript ideas Days 1–10 didn't cover

COMING FROM

KnockoutJS / .NET (C#) → moving to React + TypeScript

WHAT THIS IS

A catch-up guide filling the gaps after the 10-day plan — checked against the React roadmap (roadmap.sh/react) — so you can plan your practice around what's still missing.

How to use this: Read **Section 1 (Learning Plan)** — four parts: JavaScript, React, TypeScript-with-React, and Ecosystem — each idea with a small example. Then read **Section 2 (Detailed Notes)** slowly for the highest-value ones. Keep **Section 3 (Common Mistakes)** handy while you practise, and copy **Section 4 (Concept Notes)** by pen for recall & interviews.

1 Learning Plan — What Was Missing

These are the important ideas that Days 1–10 did not cover in depth — pulled together as a bonus round-out and cross-checked against the official React roadmap (roadmap.sh/react). They are grouped into four parts: **(1)** extra JavaScript essentials, **(2)** React features we skipped, **(3)** TypeScript-with-React must-knows, and **(4)** the wider ecosystem (testing, styling, accessibility, frameworks). Skim the part you need, or read it all before your next practice project.

Part 1 — JavaScript Essentials Not Yet Covered

You already write modern JS (Day 1) and TypeScript (Day 2). This part fills the deeper JavaScript gaps that quietly power React. Read each one, type the snippet yourself, and lean on the C#/.NET comparisons — they map cleanly.

Closures

A closure is a function that *remembers* the variables from where it was created, even after that outer function has finished running. The inner function keeps a live link to those variables (not a copy), which is how you make private state that survives between calls.

```
function makeCounter() {  
  let count = 0;           // stays alive via the closure  
  return () => ++count;  
}  
const next = makeCounter();  
next(); // 1  
next(); // 2
```

React trap: an event handler or effect can "capture" an old state value and keep using it after a re-render — that is the classic *stale closure*. **.NET / Knockout:** exactly like a C# lambda capturing a local variable — the variable lives as long as the lambda does. **Why it matters:** you'll understand how `useState`, custom hooks, and handlers hold onto values between renders.

Lexical Scope, Hoisting & the Temporal Dead Zone

Lexical scope means a variable is visible based on *where you wrote it*, not where it is called from. `var` is "hoisted" (moved to the top and set to `undefined`), so using it early gives `undefined`. `let` / `const` are also hoisted but locked in a "Temporal Dead Zone" (TDZ) until their line runs, so using them early throws — and they are block-scoped (`{ }`), while `var` is function-scoped.

```
console.log(a); // undefined - var is hoisted, then assigned
var a = 1;

// console.log(b); // ReferenceError - b is in the "Temporal Dead Zone"
let b = 2;
console.log(b); // 2
```

.NET / Knockout: C# already blocks use-before-declaration and scopes to blocks — `let` / `const` finally bring JS in line with that. **Why it matters:** you'll know why to always use `const` / `let`, never `var`, and why "used before declaration" errors happen.

The `this` Keyword + `call` / `apply` / `bind`

In a regular function, `this` is decided by *how* the function is called, not where it was defined — so it can change unexpectedly. `call` and `apply` run a function while you set `this` yourself; `bind` returns a new function with `this` locked in. Arrow functions have no own `this` (they borrow it from the surrounding code), which killed a whole category of bugs.

```
function greet(greeting) {
  return `${greeting}, ${this.name}`;
}

const user = { name: "Ada" };
greet.call(user, "Hi");           // "Hi, Ada" - call: args listed
greet.apply(user, ["Hey"]);       // "Hey, Ada" - apply: args as array
const bound = greet.bind(user);
bound("Yo");                      // "Yo, Ada" - bind: returns a new fn
```

.NET / Knockout: unlike C# where `this` always means the current instance, JS `this` is set at call time — arrow functions restore the C#-like feel. **Why it matters:** you'll avoid "this is undefined" errors and know why handlers are usually arrow functions.

Prototypes & Prototypal Inheritance

Every JS object has a hidden link to another object called its *prototype*. When you read a property the object does not have, JS walks up this "prototype chain" until it finds it. This is how objects share methods — and it is the machinery behind JS `class`.

```
const animal = {
  speak() {
    return `${this.name} makes a sound`;
  },
};
const dog = Object.create(animal); // dog's prototype is animal
dog.name = "Rex";
dog.speak(); // "Rex makes a sound" - found up the prototype chain
```

.NET / Knockout: it plays the role base classes play in C# — shared behaviour lives "up the chain" instead of on each object. **Why it matters:** you'll read library code confidently and understand what a `class` really is in JS.

The Event Loop — Microtasks vs Macrotasks

JavaScript runs on one thread with one call stack. Slow work (timers, `fetch`) is handed off to the browser, and its callback waits in a queue when done. Between tasks the engine first drains *all* microtasks (Promise `.then`, `await` continuations), then takes one macrotask (`setTimeout`, events) — that is why a resolved Promise runs before a `setTimeout(0)`.

```
console.log("1");
setTimeout(() => console.log("4"), 0); // macrotask
Promise.resolve().then(() => console.log("3")); // microtask
console.log("2");
// Order printed: 1, 2, 3, 4
```

.NET / Knockout: microtasks vs macrotasks are like how `async` / `await` continuations get scheduled ahead of lower-priority queued work. **Why it matters:** you'll understand the exact order state updates, effects, and awaited data resolve in.

Equality & Type Coercion

`==` compares *after* silently converting types (coercion), which causes surprises; `===` compares value **and** type with no conversion — always prefer it. "Falsy" values (`false`, `0`, `""`, `null`, `undefined`, `NaN`) act like `false` in conditions; everything else is "truthy".

```
0 == ""; // true - == coerces types before comparing
0 === ""; // false - === also checks the type
null == undefined; // true
NaN === NaN; // false - NaN never equals itself

const label = "" || "Guest"; // "Guest" - fall back when left side is falsy
```

.NET / Knockout: `===` is the strict, predictable comparison you already expect from C#'s `==`. **Why it matters:** you'll dodge subtle bugs in conditions, guards, and default values.

Promise Combinators

These run several promises together instead of one at a time. `Promise.all` waits for all to succeed and fails fast if any rejects. `Promise.allSettled` always waits for every one and reports each result.

`Promise.race` settles as soon as the first one settles (win or lose); `Promise.any` resolves with the first *success*, ignoring failures.

```
await Promise.all([fetchA(), fetchB()]);           // all succeed, or reject fast
await Promise.allSettled([fetchA(), fetchB()]);    // every result, success or fail
await Promise.race([fetchA(), timeout(5000)]);     // first to settle wins
await Promise.any([mirror1(), mirror2()]);         // first to SUCCEED wins
```

.NET / Knockout: `Promise.all` is `Task.WhenAll`; `Promise.race` is `Task.WhenAny`. **Why it matters:** you'll fire parallel API calls in a data layer instead of awaiting them one by one.

Essential Array & Object Methods (beyond map/filter/reduce)

`find` / `findIndex` locate one item; `some` / `every` / `includes` ask yes/no questions; `flat` / `flatMap` flatten nested arrays. `Object.keys` / `values` / `entries` turn an object into arrays you can loop over, and `Object.fromEntries` builds an object back from pairs.

```
const nums = [1, 2, 3, 4];
nums.find(n => n > 2);           // 3
nums.some(n => n > 3);           // true
nums.every(n => n > 0);           // true
nums.includes(2);                // true
[[1], [2, 3]].flat();            // [1, 2, 3]
[1, 2].flatMap(n => [n, n]);     // [1, 1, 2, 2]

const user = { id: 1, name: "Ada" };
Object.entries(user);             // [["id", 1], ["name", "Ada"]]
Object.fromEntries([["a", 1]]);  // { a: 1 }
```

.NET / Knockout: these mirror LINQ — `find` $\approx$ `FirstOrDefault`, `some` $\approx$ `Any`, `every` $\approx$ `All`, `includes` $\approx$ `Contains`. **Why it matters:** you'll reshape props and API data in one readable line instead of manual loops.

Pure Functions & Immutability

A *pure* function returns the same output for the same input and changes nothing outside itself (no side effects). *Immutability* means you never edit an existing object/array — you make a new copy with the

change. React compares old vs new references to decide what to re-render, so it depends on you not mutating.

```
// Pure: same input → same output, no side effects
const add = (a, b) ⇒ a + b;

// Immutable update: copy, don't mutate
const state = { count: 1, tags: ["a"] };
const next = { ...state, count: state.count + 1 };
const clone = structuredClone(state); // full independent deep copy
```

.NET / Knockout: like a static method with no shared state, plus C# records and `with` expressions for copy-on-change. **Why it matters:** this is the #1 rule for correct `useState` updates and predictable renders.

Map & Set

A `Map` is a key/value store where keys can be *any* type (even objects), with an easy `.size` and clean iteration. A `Set` stores only unique values, so it is perfect for de-duping. Reach for these when a plain object feels awkward.

```
const seen = new Set([1, 1, 2, 3]);
seen.has(2); // true
seen.size;   // 3 – duplicates dropped

const userObj = { id: 1 };
const cache = new Map();
cache.set("key", 42);
cache.set(userObj, "objects work as keys!");
cache.get("key"); // 42
cache.size;       // 2
```

.NET / Knockout: `Map` is `Dictionary<TKey, TValue>`; `Set` is `HashSet<T>`. **Why it matters:** you'll de-dupe lists and build lookups without hacks like object keys becoming strings.

Debounce & Throttle

Both limit how often a function runs. *Debounce* waits until the calls stop for a set time, then runs once — great for a search box. *Throttle* runs at most once per interval — great for scroll or resize. They keep expensive work (API calls, re-renders) under control.

```
function debounce(fn, delay) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}

const search = debounce((q) => console.log("search:", q), 300);
```

.NET / Knockout: similar to coalescing rapid events with a timer/ `CancellationToken` before doing the real work. **Why it matters:** you'll stop firing an API request on every keystroke.

Part 2 — React Concepts Not Yet Covered

You finished the 10 days, so you already write real React. This part fills the gaps the roadmap expects you to know — small tools you will meet in day-to-day code and in code reviews.

Refs

useRef

`useRef` gives you a small box with a `.current` property that stays the same object across every re-render. It has two jobs: (1) hold a reference to a DOM node so you can call browser methods on it (focus, scroll, measure); (2) store a value that survives re-renders but does **not** cause a re-render when you change it. Remember the split: changing state re-renders the UI; changing a ref does not.

```
import { useRef, useEffect } from "react";

function SearchBox() {
  const inputRef = useRef<HTMLInputElement>(null); // (1) a DOM node
  const renderCount = useRef(0); // (2) a value that survives renders

  useEffect(() => {
    inputRef.current?.focus(); // focus the input after it mounts
    renderCount.current += 1; // changing this does NOT re-render
  });

  return <input ref={inputRef} placeholder="Search ... " />;
}
```

.NET / Knockout: the mutable-value form is like a private field on a class — it keeps its value between calls but changing it does not repaint the screen; a Knockout `observable` would, a plain field would not.

Why it matters: you can focus/scroll/measure real DOM elements, and stash things like timer ids without triggering extra renders.

Refs to components (forwardRef and the React 19 ref-as-prop)

Sometimes a parent needs the DOM node inside a *child* component. Before React 19 you had to wrap the child in `forwardRef` to receive a `ref`. In React 19 that boilerplate is gone: `ref` is just a normal prop you can type and destructure. If you want the child to expose *methods* instead of the raw node, `useImperativeHandle` lets it hand the parent a small custom object like `{ focus() {...} }`.

```
import type { Ref } from "react";

interface Props {
  ref?: Ref<HTMLInputElement>;
  label: string;
}

function TextField({ ref, label }: Props) {
  return (
    <label>
      {label}
      <input ref={ref} />
    </label>
  );
}

// Parent: <TextField ref={myRef} label="Name" />
```

.NET / Knockout: like exposing a public method on a user control so the parent screen can call it; `useImperativeHandle` ~ defining that small public surface deliberately.

Why it matters: you can build reusable inputs a parent can focus or scroll to — with no `forwardRef` wrapper in React 19.

Rendering structure

Fragments

A component must return one parent element, but you often do not want an extra `<div>` in the DOM. A Fragment groups siblings without adding any wrapper — write the empty tag `<...>`. When you render a Fragment inside a list and need a `key`, use the full form `<React.Fragment key={id}>...</React.Fragment>` (the only place the short syntax cannot take a key).


```
function Row({ label, value }: { label: string; value: string }) {
  return (
    <
      <dt>{label}</dt>
      <dd>{value}</dd>
    </>
  );
}
```

.NET / Knockout: like Knockout's containerless `<!-- ko -->` binding — you output several nodes without a wrapper element around them.

Why it matters: clean, valid HTML (table rows, `<dl>` pairs) and layouts that do not break because of an unwanted wrapper div.

Portals

`createPortal` renders children into a *different* place in the real DOM (usually `document.body`) while keeping them in your React tree — so props, context, state, and events still flow exactly as if they were nested where you wrote them. This is the standard fix for modals, tooltips, and toasts that must escape a parent's `overflow: hidden` or `z-index`.

```
import { createPortal } from "react-dom";
import type { ReactNode } from "react";

function Modal({ children }: { children: ReactNode }) {
  return createPortal(
    <div className="overlay">{children}</div>,
    document.body
  );
}
```

.NET / Knockout: like rendering a dialog at the window root instead of inside the current panel, while it still logically belongs to the same view.

Why it matters: modals and tooltips that are never clipped or stacked wrong, yet still receive the context and props from where you declared them.

StrictMode

`<StrictMode>` is a dev-only wrapper. In development it intentionally runs some functions twice — your component body, and each effect's setup-then-cleanup-then-setup — to surface bugs like effects with missing cleanup or impure render logic. It does nothing in production. This is why you sometimes see an effect or a `console.log` fire twice while developing.

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "../App";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

.NET / Knockout: like extra Debug-build assertions or analyzers that catch mistakes during development and are stripped from the Release build.

Why it matters: it demystifies the "runs twice" behavior and pushes you to write effects with correct cleanup.

Timing and ids

useLayoutEffect vs useEffect

Both run after render, but the timing differs. `useEffect` runs *after* the browser paints (use it for data fetching, subscriptions, most things). `useLayoutEffect` runs *synchronously before* the paint, so the user never sees an intermediate frame — use it only when you must measure layout and change the DOM in the same beat to avoid a visible flicker. Default to `useEffect`; reach for `useLayoutEffect` rarely.

```
import { useLayoutEffect, useRef, useState } from "react";

function Tooltip() {
  const ref = useRef<HTMLDivElement>(null);
  const [height, setHeight] = useState(0);

  useLayoutEffect(() => {
    setHeight(ref.current!.offsetHeight); // measure BEFORE paint = no flicker
  }, []);

  return <div ref={ref}>Height: {height}px</div>;
}
```

.NET / Knockout: `useEffect` ~ posting work to run after the UI thread finishes painting;
`useLayoutEffect` ~ doing it inside the same synchronous layout pass, before anything is shown.

Why it matters: you can kill flicker when positioning tooltips or reading element sizes — without overusing a blocking effect.

useId

`useId` returns a stable, unique string id that matches between server and client (SSR-safe). Use it to connect a `<label>` to its `<input>` via `htmlFor` / `id`, or for `aria-*` attributes. Do not use it for list keys — that is not its job.

```
import { useId } from "react";

function EmailField() {
  const id = useId();
  return (
    <
      <label htmlFor={id}>Email</label>
      <input id={id} type="email" />
    >
  );
}
```

.NET / Knockout: like an auto-generated unique `ClientID` so the label's `for` matches the input — but collision-free and done for you.

Why it matters: accessible, correctly paired form fields inside reusable components, with no hand-managed id strings.

Suspense and concurrent UI

Suspense

`<Suspense>` shows a `fallback` (like a spinner) while something inside it is still loading. The most common trigger is a lazy-loaded component (`React.lazy`, which you met on Day 8); Suspense-enabled data libraries and the `use()` hook can also trigger it. You wrap the part that might "suspend" and give it a fallback UI.

```
import { Suspense, lazy } from "react";

const Dashboard = lazy(() => import("./Dashboard"));

function App() {
  return (
    <Suspense fallback={<p>Loading ... </p>}>
      <Dashboard />
    </Suspense>
  );
}
```

.NET / Knockout: like an async view that shows a placeholder until the awaited content is ready — declared once, instead of manual `isLoading` flags everywhere.

Why it matters: one clean loading boundary for lazy screens (and data), instead of scattering `isLoading` checks across components.

useTransition and useDeferredValue

These keep the UI responsive during heavy updates by marking some work as "not urgent."

`useTransition` gives you `const [isPending, startTransition] = useTransition();` — wrap a slow state update in `startTransition(() => setTab(next))` so React can interrupt it and keep typing/clicks snappy, and use `isPending` to show a subtle spinner. `useDeferredValue` takes a value and returns a slightly-lagging copy that updates at low priority — ideal for filtering a large list while the input stays smooth.

```
import { useDeferredValue, useState } from "react";

function Search({ items }: { items: string[] }) {
  const [query, setQuery] = useState("");
  const deferredQuery = useDeferredValue(query); // low-priority copy
  const results = items.filter((i) => i.includes(deferredQuery));

  return (
    <div>
      <input value={query} onChange={(e) => setQuery(e.target.value)} />
      <ul>{results.map((r) => <li key={r}>{r}</li>)}</ul>
    </div>
  );
}
```

.NET / Knockout: like scheduling heavy work at lower thread priority (or debouncing) so the input stays responsive — but built into rendering, no manual timer.

Why it matters: the text box stays smooth even while filtering thousands of rows, with no hand-rolled debounce.

React 19 Actions and the use() hook

Form Actions with useActionState

In React 19 a `<form>` can take an `action` function that receives the submitted `FormData` directly — no `onSubmit`, no `preventDefault`. `useActionState` wraps that action to track the latest returned result plus a `pending` flag. React runs your action, marks the form pending while it awaits, and stores the return value as the new state.

```
import { useActionState } from "react";

function NameForm() {
  const [message, submit, isPending] = useActionState(
    async (_prev: string, formData: FormData) => {
      const name = formData.get("name") as string;
      return `Saved ${name}`;
    },
    ""
  );

  return (
    <form action={submit}>
      <input name="name" />
      <button disabled={isPending}>Save</button>
      {message} && <p>{message}</p>
    </form>
  );
}
```

.NET / Knockout: like an MVC controller action bound to a form post that returns a result model — but client-side, with the pending state handled for you.

Why it matters: submit + pending + result state come from one hook, replacing a pile of manual `useState` flags.

useFormStatus

`useFormStatus` lets a component read the pending state of the nearest parent `<form>` — without passing props down. It must run in a component rendered *inside* the form. This is the clean way to build a

reusable Submit button that disables itself while its form is submitting.

```
import { useFormStatus } from "react-dom";

function SubmitButton() {
  const { pending } = useFormStatus();
  return <button disabled={pending}>{pending ? "Saving ... " : "Save"}</button>;
}
```

.NET / Knockout: like a shared button control that automatically knows the surrounding form is posting back, without wiring a flag through to it.

Why it matters: one reusable submit button that "just knows" when its form is busy — drop it into any form.

useOptimistic

`useOptimistic` shows an instant, guessed result while the real async action is still running, then reconciles with the true state when it settles. You render the optimistic value; when the action finishes (or the state updates), React swaps in the confirmed value. Trigger the optimistic update inside an action or `startTransition`.

```
import { useOptimistic, useState, startTransition } from "react";

function Likes() {
  const [likes, setLikes] = useState(0);
  const [optimistic, addOptimistic] = useOptimistic(likes);

  function like() {
    startTransition(async () => {
      addOptimistic(optimistic + 1); // show +1 instantly
      await new Promise((r) => setTimeout(r, 500));
      setLikes((n) => n + 1); // confirm the real value
    });
  }

  return <button onClick={like}>Likes: {optimistic}</button>;
}
```

.NET / Knockout: like updating the UI immediately and rolling back if the server call fails — optimistic concurrency, but for the view.

Why it matters: snappy interactions (likes, todos, toggles) that feel instant while the network catches up.

The use() hook

`use()` reads the value of a promise or a context *during render*. Give it a promise and the component suspends until it resolves (pair it with `<Suspense>`); give it a context and it reads the value like `useContext`. Unlike every other hook, `use()` may be called conditionally — inside an `if` or a loop.

```
import { use } from "react";

function Message({ promise }: { promise: Promise<string> }) {
  const text = use(promise); // suspends until the promise resolves
  return <p>{text}</p>;
}

// Render inside <Suspense fallback={<Spinner />}>
```

.NET / Knockout: like `await` on a `Task`, but declarative inside render — React suspends the component instead of blocking a thread.

Why it matters: you can unwrap async data or read context in one line and let Suspense own the loading UI.

Data fetching and state libraries

TanStack Query (React Query)

TanStack Query manages *server* data for you. Instead of hand-writing `useEffect` + `useState` + loading/error (your Day 5 pattern), `useQuery` gives you caching, request de-duplication, automatic retries, background refetch, and ready-made `isLoading` / `error` / `data`. `useMutation` does the same for writes (POST/PUT/DELETE). You wrap your app once in `<QueryClientProvider client={new QueryClient()}>`.

```
import { useQuery } from "@tanstack/react-query";

interface User {
  id: number;
  name: string;
}

function Users() {
  const { data, isLoading, error } = useQuery({
    queryKey: ["users"],
    queryFn: (): Promise<User[]> =>
      fetch("/api/users").then((r) => r.json()),
  });

  if (isLoading) return <p>Loading ... </p>;
  if (error) return <p>Failed to load</p>;
  return <ul>{data?.map((u) => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

.NET / Knockout: like a caching data layer — `MemoryCache` plus a repository — that also handles staleness, retries, and refetching for you.

Why it matters: you can delete most of your `useEffect` fetching code and get caching, dedupe, and refetch for free.

Zustand

For *client* state shared across many components, a dedicated store is simpler than threading Context everywhere. Zustand is minimal: `create()` takes a function returning state plus updater functions and hands you back a hook. Any component reads just the slice it needs (via a selector) and only re-renders when that slice changes — no provider required.


```
import { create } from "zustand";

interface CounterState {
  count: number;
  increment: () => void;
}

export const useCounter = create<CounterState>((set) => ({
  count: 0,
  increment: () => set((s) => ({ count: s.count + 1 })),
}));

// In a component: const count = useCounter((s) => s.count);
```

.NET / Knockout: like a lightweight singleton service holding state, used wherever needed — but with automatic change notifications to the UI.

Why it matters: share state across the tree with almost no boilerplate and no provider wrapping.

Redux Toolkit

Redux Toolkit (RTK) is the modern, official Redux — the structured choice for large apps. `createSlice` bundles state, action creators, and reducer in one place, with safe "mutating" syntax powered by Immer. You configure one store and read/write it through typed `useSelector` / `useDispatch` hooks. **Rule of thumb:** Context for small, rarely-changing values (theme, current user); Zustand for simple shared state; Redux Toolkit for large apps with complex, frequently-changing state and devtools needs.

```
import { createSlice, configureStore } from "@reduxjs/toolkit";

const counter = createSlice({
  name: "counter",
  initialState: { value: 0 },
  reducers: {
    increment: (state) => {
      state.value += 1; // Immer makes this "mutation" safe
    },
  },
});

export const { increment } = counter.actions;
export const store = configureStore({
  reducer: { counter: counter.reducer },
});

export type RootState = ReturnType<typeof store.getState>;
```

.NET / Knockout: like a central, predictable state container with a single dispatch pipeline — think CQRS / MediatR commands flowing through one place, plus time-travel devtools.

Why it matters: predictable, debuggable state for large apps, with typed selectors keeping reads safe.

Composition

Component composition

React favors *composition over configuration*: instead of one component with dozens of boolean props, you pass smaller components *inside* it. The special `children` prop renders whatever a parent nests between your tags; you can also accept elements as named props to build "slots." Keep components small and combine them.

```
import type { ReactNode } from "react";

function Card({ title, children }: { title: string; children: ReactNode }) {
  return (
    <section className="card">
      <h2>{title}</h2>
      {children}
    </section>
  );
}

// <Card title="Profile"><Avatar /><Bio /></Card>
```

.NET / Knockout: like a Razor layout with `@RenderBody()`, or a Blazor component taking a `RenderFragment` child — you slot content in rather than flipping flags.

Why it matters: flexible, reusable components without prop explosion. Heads-up: older codebases still use Higher-Order Components (HOCs) and render props for reuse — hooks have largely replaced both, but you will recognize them in review.

Part 3 — TypeScript-with-React Essentials

Utility types

Utility types are built-in "type helpers" that build a new type from one you already have — so you never re-type a shape by hand. The most useful ones: `Partial` (all fields optional), `Required`, `Readonly`, `Pick` (keep some fields), `Omit` (drop some fields), `Record` (a keyed lookup map), `ReturnType` (the type a function returns), and `NonNullable` (strip `null` / `undefined`). These shine on React props and state — e.g. `Omit` an `id` when creating a record, or `Partial` for an "edit patch".

```

interface User {
  id: number;
  name: string;
  email: string;
}

type NewUser = Omit<User, 'id'>; // creating: no id yet
type RoleCounts = Record<'admin' | 'user', number>; // a lookup map
type UserPatch = Partial<User>; // every field optional

const draft: NewUser = { name: 'Ada', email: 'ada@x.io' };
const counts: RoleCounts = { admin: 3, user: 12 };
const patch: UserPatch = { name: 'Ada L.' };

function getUser() {
  return { id: 1, name: 'Ada' };
}

type Fetched = ReturnType<typeof getUser>; // { id: number; name: string }
const user: Fetched = { id: 1, name: 'Ada' };

console.log(draft, counts, patch, user);

```

- **.NET contrast:** Like making a DTO variant in C# (a "create" DTO without the `Id`) — but you get it for free instead of hand-writing a new class.
- **Why it matters:** One source-of-truth type, many safe views of it — less duplication, fewer drifting types.

Discriminated unions for state

A discriminated union is a set of shapes joined by `|`, where each shape carries a shared literal "tag" field (here `status`). When you check the tag, TypeScript automatically narrows to that exact shape and only lets you touch the fields that exist there. Model async state this way instead of juggling separate `isLoading` / `isError` / `data` booleans — impossible combinations (loading **and** error at once) simply can't be built.

```

type UserState =
  | { status: 'idle' }
  | { status: 'loading' }
  | { status: 'success'; data: string[] }
  | { status: 'error'; message: string };

function render(state: UserState): string {
  switch (state.status) {
    case 'success':
      return state.data.join(', '); // data exists only here
    case 'error':
      return state.message;          // message exists only here
    default:
      return 'waiting ... ';
  }
}

```

- **.NET contrast:** A sealed class hierarchy + a `switch` pattern-match on the subtype — the compiler enforces you handle the right members.
- **Why it matters:** Your UI states become airtight; you can't read `data` before it has loaded.

Typing React itself

React ships its own types for the "React shapes". `ReactNode` = anything renderable (string, number, element, array, `null`); `ReactElement` = specifically a JSX element. `PropsWithChildren<T>` adds a typed `children` for you. Event handlers use typed event args like `ChangeEvent<HTMLInputElement>` (or `MouseEvent` / `FormEvent`), and a DOM ref is `useRef<HTMLInputElement>(null)`.

```

import { useRef } from 'react';
import type { PropsWithChildren, ReactNode, ChangeEvent } from 'react';

type CardProps = PropsWithChildren<{ title: string; footer?: ReactNode }>;

function Card({ title, footer, children }: CardProps) {
  const inputRef = useRef<HTMLInputElement>(null);

  function handleChange(e: ChangeEvent<HTMLInputElement>) {
    console.log(e.target.value, inputRef.current);
  }

  return (
    <section>
      <h2>{title}</h2>
      <input ref={inputRef} onChange={handleChange} />
      {children}
      {footer}
    </section>
  );
}

```

With a namespace import (`import * as React`) these read as `React.ChangeEvent< ... >`, `React.MouseEvent<HTMLButtonElement>`, `React.FormEvent<HTMLFormElement>` — same types, different spelling. For a `useReducer`, type the actions as a discriminated union too:

```

type Action =
  | { type: 'increment' }
  | { type: 'add'; amount: number }
  | { type: 'reset' };

function reducer(count: number, action: Action): number {
  switch (action.type) {
    case 'increment': return count + 1;
    case 'add':       return count + action.amount; // amount typed here
    case 'reset':     return 0;
  }
}

```

- **.NET contrast:** The event arg (`ChangeEvent`) is like an `EventArgs` payload; `useRef` is like a private field that survives across renders but does **not** trigger a UI update when changed.

- **You'll be able to:** Type any prop, handler, ref, or reducer without fighting red squiggles.

unknown vs any

`any` switches type-checking **off** — the compiler stops helping and any mistake slips through. `unknown` is the honest "I don't know this type yet" box: you can hold anything, but you must **narrow** it (prove what it is) before using it. Narrow with `typeof`, the `in` operator, or a custom type-guard function whose return type is `x is SomeType`.

```
interface Dog { bark: () => void }

// custom "is" predicate: teaches the compiler what x is
function isDog(x: unknown): x is Dog {
  return typeof x === 'object' && x !== null && 'bark' in x;
}

function speak(x: unknown): void {
  if (typeof x === 'string') console.log(x); // narrowed to string
  else if (isDog(x)) x.bark();               // narrowed to Dog
}
```

- **.NET contrast:** `unknown` is like `object` where you **must** check/cast before use; `any` is like `dynamic` — no safety net.
- **Why it matters:** Use `unknown` for API responses and caught errors so the compiler forces you to validate them.

Type assertions (`as`) and the non-null `!`

`as` tells the compiler "trust me, this is that type" and `!` tells it "this is definitely not null/undefined". Both are compile-time-only — there is **no runtime check**, so a wrong assertion crashes later. Use them sparingly: only when you truly know more than the compiler (e.g. a DOM node you just created). Prefer real narrowing (type guards) whenever you can.

```
const el = document.getElementById('root') as HTMLDivElement;
el.classList.add('ready'); // `as` removed the `| null` for us

const size = new Map<string, number>().get('x')!;
console.log(size.toFixed()); // `!` claims "not undefined" - risky if it lies
```

- **.NET contrast:** `as` is like a C# cast but **without** the runtime `InvalidCastException` safety; `!` is literally C#'s null-forgiving `!` operator.
- **Why it matters:** Overusing `as` / `!` quietly disables the very checks you turned TypeScript on for.

`as const` and the `satisfies` operator

`as const` freezes a literal to its narrowest, read-only type — an array becomes a fixed tuple of exact string literals, perfect for deriving a union type. `satisfies` checks that a value matches a target type **without widening it**, so you keep the precise inferred types (and get key autocomplete) while still being type-checked.

```
// as const: freeze to the narrowest literal type (readonly)
const ROLES = ['admin', 'user', 'guest'] as const;
type Role = (typeof ROLES)[number]; // 'admin' | 'user' | 'guest'
const current: Role = 'admin';

// satisfies: verify the shape, but keep the precise inferred types
const config = {
  port: 3000,
  host: 'localhost',
} satisfies Record<string, string | number>;

const p: number = config.port; // OK - port stays `number`, not string | number
console.log(current, p);
```

- **.NET contrast:** `as const` is like a compile-time `readonly` constant; `satisfies` is like asking the compiler "does this match the interface?" without upcasting and losing the concrete type.
- **Why it matters:** Derive unions from one array (no duplicated string lists), and validate config/theme objects without giving up their exact types.

Part 4 — Ecosystem & Professional Practices

Testing — Vitest + React Testing Library + user-event

The golden rule: **test behaviour, not implementation**. Don't poke at internal state or hook internals — instead render the component, find things the way a user would (by role or visible text), act like a user (`user-event` clicks/types), and assert on what the screen shows. That keeps tests green when you refactor internals but red when the user experience actually breaks.

```
import '@testing-library/jest-dom';
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { expect, test } from 'vitest';
import { useState } from 'react';

function Counter() {
  const [n, setN] = useState(0);
  return <button onClick={() => setN(n + 1)}>Count: {n}</button>;
}

test('increments when the user clicks', async () => {
  const user = userEvent.setup();
  render(<Counter />);
  await user.click(screen.getByRole('button', { name: /count/i }));
  expect(screen.getByText('Count: 1')).toBeInTheDocument();
});
```

For full end-to-end flows across real pages (login, navigation, forms), reach for **Playwright** or **Cypress**, which drive an actual browser.

- **.NET contrast:** Vitest + RTL is like xUnit/NUnit plus a UI harness that clicks the rendered component — assertions on the output, not the internals.
- **You'll be able to:** Ship refactors confidently, because tests break only when real behaviour changes.

Styling approaches

Three common paths: **plain CSS / CSS Modules** (write real CSS; Modules auto-scope class names so they can't clash), **Tailwind CSS** (compose tiny utility classes right in `className`), and **CSS-in-JS** like styled-components (styles live in your component file). Rough guide: CSS Modules for a small/standard app, Tailwind for fast consistent UI at scale, CSS-in-JS when styles need to react heavily to props.

```
function Badge() {
  return (
    <span className="inline-block rounded bg-blue-600 px-2 py-1 text-sm text-white">
      New
    </span>
  );
}
```

One-line styled-components note: it lets you write `const Button = styled.button\`padding: 8px 16px;`, giving you a real`

` component whose CSS is co-located and can vary by props.

- **.NET contrast:** CSS Modules' auto-scoping is like scoped component styles — no accidental global bleed between pages.
- **Why it matters:** Pick one approach per project and stay consistent; mixing three styling systems is a maintenance headache.

Accessibility (a11y) basics

Accessibility means everyone — including keyboard-only and screen-reader users — can use your app. Most of it is free if you use **semantic HTML** (`<button>` , `<nav>` , `<label>`), give inputs real labels, add `alt` text to images, keep a visible keyboard focus, and add `aria-*` only when native HTML can't express the meaning.

```
function EmailField() {
  return (
    <div>
      <label htmlFor="email">Email</label>
      <input id="email" type="email" name="email" />
      
    </div>
  );
}
```

- **.NET contrast:** `htmlFor` / `id` pairing is like a Web Forms `Label` with `AssociatedControlID` — it wires the label to its control.
- **Quick win:** Always connect `<label htmlFor="x">` to `<input id="x">` ; clicking the label focuses the field and screen readers announce it.

Frameworks & rendering

Your Vite app is an **SPA** (Single-Page App): the browser downloads JS, then React builds the whole page on the client. Alternatives: **SSR** (Server-Side Rendering) builds HTML on the server per request for fast first paint and SEO; **SSG** (Static Site Generation) pre-builds HTML at build time for content that rarely changes; **RSC** (React Server Components) render some components entirely on the server and send zero JS for them. Graduate to **Next.js** or **Remix** when you need SEO, server-side data fetching, or fast first loads — they bundle routing, SSR/SSG, and RSC for you.

- **.NET contrast:** SSR/SSG is like Razor rendering HTML on the server; an SPA is client-only rendering.
- **Note:** You probably don't need this yet for an internal app — but know it exists, so you recognise when a project has outgrown a plain SPA.

Detailed Notes — Read Once, Understand Everything

The big idea first

Your 10-day plan gave you a solid working base. You can build components, hold state, fetch data, route, validate forms, and split code. That is real React. Well done.

This section is different. It is about the *why* underneath. These are the ideas that separate someone who "makes React work" from someone who "understands React". When something breaks at 5pm, the first person guesses; the second person knows exactly where to look.

We will not list a hundred things. We go deep on the few that pay off the most, and give the rest an honest one-liner so you know they exist.

1. Closures and the "stale closure" trap

A **closure** is just a function that remembers the variables that were around when it was created. In C# terms: it is exactly a lambda that *captures* a local variable. The lambda keeps a handle on that variable even after the outer method returns.

```
// C# - the lambda "closes over" message
Func<string> greet() {
    var message = "hi";
    return () => message; // remembers message
}
```

JavaScript does the same thing everywhere, and React leans on it hard. Every render of your component creates *fresh* functions that capture the state values *from that render*. This is the whole trap.

Look at this timer. It only ever counts to 1, then sits at 1 forever:

```
import { useState, useEffect } from 'react';

export function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const id = setInterval(() => {
      // Stale closure: setCount(count + 1) would always read count = 0.
      // Fix: functional update always gets the latest value.
      setCount(c => c + 1);
    }, 1000);
    return () => clearInterval(id);
  }, []); // empty deps: closure captures count = 0 forever

  return <p>{count}</p>;
}
```

Here is *why* the broken version fails. The effect runs once (empty dependency array `[]`). At that moment `count` is `0`. The arrow function inside `setInterval` captures that `0` — just like a C# lambda capturing a local. Every tick, one second apart, it computes `0 + 1` and sets `count` to `1`. It never sees a newer value, because it is frozen in the first render's world.

There are two clean fixes:

1. **Functional update** — `setCount(c => c + 1)`. Instead of reading the captured `count`, you ask React "whatever the current value is, add one". React always passes the freshest value. This is the fix shown above and the one to reach for most.
2. **Correct dependency arrays** — if the effect really needs the latest `count`, list it: `{, [count]}`. Now the effect re-runs each time `count` changes and re-captures the new value. The catch: for a subscription like `setInterval`, that tears down and recreates the interval every second, which is wasteful. When you want "read the latest value without re-subscribing", use a `useRef` (next section).

The ESLint rule `react-hooks/exhaustive-deps` exists to warn you when your dependency array is lying about what the effect actually uses. Do not silence it blindly — a missing dependency is almost always a stale closure waiting to bite.

2. The event loop, in one clear mental model

You already use `async` / `await`. This is what runs underneath. In .NET, `await` schedules a *continuation* to run later on a scheduler; JavaScript has a similar idea but with one single thread and a fixed set of queues. Once you can see the queues, `async` stops being magic.

Four parts:

- **Call stack** — where your code runs right now. One thing at a time. Single-threaded.
- **Web APIs** — the browser (or Node) handles slow work off to the side: timers, network, DOM events. Your thread is not blocked while these run.
- **Microtask queue** — resolved `Promise` callbacks (`.then` , and the code after an `await`) go here.
- **Macrotask queue** — `setTimeout` , `setInterval` , UI events go here.

The rule that explains everything: the call stack runs to empty, then the loop **drains the entire microtask queue**, and only then takes **one** macrotask. So promises always jump ahead of timers.

```
console.log('1: start');

setTimeout(() => console.log('4: setTimeout (macrotask)'), 0);

Promise.resolve().then(() => console.log('3: promise (microtask)'));

console.log('2: end');

// Output order: 1, 2, 3, 4
```

Even with a `0` ms timeout, the promise wins. `1` and `2` are plain synchronous code on the stack. When the stack empties, microtasks run (`3`), then finally the macrotask (`4`).

Why you care in React: this is why a state update inside a `.then` or after an `await` lands *after* the current synchronous code finishes, and why a tight loop that never yields will freeze the UI — nothing else can run until the stack is clear.

3. useRef, properly

`useRef` gives you a small box: `{ current: ... }`. It has two jobs, and mixing them up causes confusion.

The cleanest C# comparison: a `useRef` value is like a **private field on an object that survives across renders but does not trigger a UI update when you change it**. `useState` is the opposite: changing it *does* re-render.

Job 1 — a handle to a DOM element. Sometimes you need the real DOM node to call something imperative like `.focus()`.

```
import { useRef } from 'react';

export function SearchBox() {
  const inputRef = useRef<HTMLInputElement>(null);
  const focusInput = () => {
    inputRef.current?.focus();
  };
  return (
    <div>
      <input ref={inputRef} type="text" />
      <button onClick={focusInput}>Focus</button>
    </div>
  );
}
```

You type the ref as `useRef<HTMLInputElement>(null)`. React fills `inputRef.current` with the real `<input>` after it mounts. The `?.` guards the brief moment it is still `null`.

Job 2 — a mutable "instance variable" that must NOT re-render. This is the fix for the subscription problem from section 1: keep the latest value in a ref so a long-lived callback can read it.

```
import { useRef, useState, useEffect } from 'react';

export function Ticker() {
  const [count, setCount] = useState(0);
  const latestCount = useRef(count);

  // After every render, point the ref at the newest value.
  useEffect(() => {
    latestCount.current = count;
  });

  useEffect(() => {
    const id = setInterval(() => {
      // Reads the newest count without re-subscribing the interval.
      console.log('latest count is', latestCount.current);
    }, 1000);
    return () => clearInterval(id);
  }, []);

  return <button onClick={() => setCount(c => c + 1)}>{count}</button>;
}
```

The interval is created once, yet always logs the current count, because it reads `latestCount.current`, and a separate effect keeps that box updated on every render.

When do you pick `useRef` over `useState`? Ask one question: *does the screen need to change when this value changes?*

- Yes, the UI depends on it → `useState`.
- No, it is just bookkeeping the render does not read — a timer id, a previous value, a "has this run before" flag, a DOM node → `useRef`. Changing a ref never re-renders, so never read `ref.current` during render to decide what to show.

(Small React 19 note: you no longer need `forwardRef` to pass a ref into your own component — `ref` can be a normal prop now. Less boilerplate than before.)

4. Model async state as one union, not three booleans

The common way to track a fetch is three separate pieces: `isLoading`, `isError`, `data`. The problem is that TypeScript then believes nonsense combinations are possible: loading *and* error at once, or `data` present while `isLoading` is still `true`. You end up writing defensive checks for states that should never exist.

A **discriminated union** fixes this at the type level. This is the TypeScript version of a **sealed class hierarchy with pattern matching** in C# — a closed set of shapes, each tagged with a discriminator you `switch` on.

```
type AsyncState<T> =
  | { status: 'idle' }
  | { status: 'loading' }
  | { status: 'success'; data: T }
  | { status: 'error'; error: string };

interface User {
  id: number;
  name: string;
}

export function UserView({ state }: { state: AsyncState<User> }) {
  switch (state.status) {
    case 'idle':
      return <p>Nothing yet.</p>;
    case 'loading':
      return <p>Loading ... </p>;
    case 'error':
      // TS knows `error` exists here; `data` does not.
      return <p>Oops: {state.error}</p>;
    case 'success':
      // TS knows `data` exists here.
      return <p>Hello {state.data.name}</p>;
  }
}
```

The magic word is `status`. Because every branch has a different literal `status`, TypeScript **narrows** the type inside each `case`. In the `'success'` branch it *knows* `data` exists; in `'error'` it *knows* `error` exists and `data` does not. Try to read `state.data` in the `error` branch and it will not compile. Impossible states become unrepresentable — the bug is caught before it can ship.

5. Why a data library (TanStack Query) exists

Hand-rolled `useEffect` fetching works for one screen. It stops being fun the moment you have many. Here is the everyday version you already know how to write:

```

import { useState, useEffect } from 'react';
import axios from 'axios';

interface User {
  id: number;
  name: string;
}

export function UserList() {
  const [users, setUsers] = useState<User[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    let cancelled = false;
    setLoading(true);
    axios
      .get<User[]>('/api/users')
      .then(res => { if (!cancelled) setUsers(res.data); })
      .catch(() => { if (!cancelled) setError('Failed to load'); })
      .finally(() => { if (!cancelled) setLoading(false); });
    return () => { cancelled = true; };
  }, []);

  if (loading) return <p>Loading ... </p>;
  if (error) return <p>{error}</p>;
  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}

```

Look at what you had to do by hand: three state variables, a `cancelled` flag to dodge a **race condition** (the request finishing after the component is gone), and no sharing — open this list on two screens and you fetch twice. There is no caching, no refetch when the user comes back to the tab, no retry. And you copy this whole dance into every component.

TanStack Query is a caching data layer. The closest C# picture: `MemoryCache` sitting in front of a repository. You describe *what* the data is (a key) and *how* to get it (a function); it handles the caching, sharing, refetching, and lifecycle.


```

import { useQuery } from '@tanstack/react-query';
import axios from 'axios';

interface User {
  id: number;
  name: string;
}

export function UserList() {
  const { data, isLoading, isError } = useQuery({
    queryKey: ['users'],
    queryFn: async () => {
      const res = await axios.get<User[]>('/api/users');
      return res.data;
    },
  });

  if (isLoading) return <p>Loading ... </p>;
  if (isError) return <p>Failed to load</p>;
  if (!data) return null;
  return <ul>{data.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}

```

Same result, and now: the `queryKey` `['users']` is the cache key — two components asking for `['users']` share one fetch and one cache entry. Race conditions are handled for you. It can refetch on window focus, retry on failure, and serve cached data instantly while quietly updating in the background. You stop writing the loading/error/cancel boilerplate and start describing data. (One-time setup: wrap your app in a `QueryClientProvider`.)

6. React 19, brief but concrete

React 19 added tools aimed squarely at forms and pending UI — the stuff you hand-wired on Day 6.

Actions + `useActionState` — a form can call an async function directly, and `useActionState` tracks the pending flag and the result for you. No manual `isSubmitting` state.

```

import { useActionState } from 'react';

interface State { message: string; }

async function submitName(_prev: State, formData: FormData): Promise<State> {
  const name = formData.get('name') as string;
  await new Promise(r => setTimeout(r, 500)); // pretend API call
  return { message: `Saved ${name}!` };
}

export function NameForm() {
  const [state, formAction, isPending] = useActionState(submitName, {
    message: '',
  });
  return (
    <form action={formAction}>
      <input name="name" />
      <button disabled={isPending}>Save</button>
      <p>{state.message}</p>
    </form>
  );
}

```

`useActionState` hands back the latest result (`state`), the action to attach to the form, and `isPending` — all managed for you. The problem it solves: you no longer thread submitting/result state by hand.

`useFormStatus` — lets a child (like a submit button) read the parent form's pending state without passing props down.

```

import { useFormStatus } from 'react-dom';

export function SubmitButton() {
  const { pending } = useFormStatus();
  return <button disabled={pending}>{pending ? 'Saving ... ' : 'Save'}</button>;
}

```

Drop `<SubmitButton />` inside any `<form>` and it disables itself while that form submits. The problem it solves: prop-drilling a `pending` flag to deep buttons.

`useOptimistic` — show the expected result *immediately*, before the server confirms, then reconcile. Great for chat, likes, to-do adds.

```

import { useOptimistic, useState } from 'react';

interface Message {
  id: number;
  text: string;
  sending?: boolean;
}

export function Chat() {
  const [messages, setMessages] = useState<Message[]>([]);
  const [optimistic, addOptimistic] = useOptimistic(
    messages,
    (current, text: string): Message[] => [
      ...current,
      { id: Date.now(), text, sending: true },
    ]
  );

  async function send(formData: FormData) {
    const text = formData.get('text') as string;
    addOptimistic(text); // show instantly
    const saved: Message = { id: Date.now(), text };
    setMessages(prev => [...prev, saved]); // real update
  }

  return (
    <form action={send}>
      <input name="text" />
      {optimistic.map(m => (
        <p key={m.id}>
          {m.text} {m.sending ? '(sending ...)' : ''}
        </p>
      ))}
    </form>
  );
}

```

The message appears the instant you send it, marked "(sending...)". When the real save finishes and `messages` updates, React drops the temporary optimistic entry. The problem it solves: laggy UIs that feel slow because they wait for the round-trip.

7. The testing mindset with React Testing Library

The one rule that makes tests worth keeping: **test what the user does and sees, not the internal state**. If you assert on `useState` values or private methods, your tests break every time you refactor, even when nothing is actually broken. If you assert on visible text and behaviour, your tests only break when the *user's* experience breaks — which is the whole point.

Think of it as **xUnit/NUnit plus a small UI harness**: you render a component, act on it like a person would (click, type), and assert on what shows up.

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { test, expect } from 'vitest';
import '@testing-library/jest-dom';
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(c => c + 1)}>Count: {count}</button>;
}

test('increments when the user clicks', async () => {
  render(<Counter />);
  const button = screen.getByRole('button', { name: /count: 0/i });
  await userEvent.click(button);
  // We assert what the user SEES, not internal state.
  expect(screen.getByRole('button', { name: /count: 1/i })).toBeInTheDocument();
});
```

Notice: we never touch `count`. We find the button *the way a user (or a screen reader) would* — by its role and its visible label — click it, and check the label changed. That test would survive a rewrite from `useState` to `useReducer` without a single edit, because the user-facing behaviour is identical. Querying by role also nudges you toward accessible markup.

8. An honest word on the rest

You do not need these yet, but you should know they exist and roughly when to reach for them:

- **Portals** — render a child into a different part of the DOM (outside the parent) while keeping it in your component tree. Use for modals, tooltips, toasts that must escape a container's `overflow` or `z-index`.
- **Suspense** — lets a component "wait" for something (lazy-loaded code, or data) and show a fallback meanwhile. You already met it via `React.lazy`; data libraries increasingly use it too.

- **useTransition** — mark a state update as low-priority so an urgent one (like typing) stays snappy while heavier work (like filtering a big list) catches up.
 - **Zustand / Redux Toolkit** — global state stores for when many far-apart components share state and Context starts feeling clumsy. Zustand is tiny and simple; Redux Toolkit is the structured, battle-tested option with great devtools. Reach for one only when you feel the pain — not before.
 - **Styling** — CSS Modules, Tailwind, or CSS-in-JS. Pick one and be consistent; none is "the React way".
 - **Accessibility (a11y)** — semantic HTML, labels, roles, keyboard support. Mostly free if you use real elements (`<button>` , `<label>`) instead of clickable `<div>` s. RTL's role-based queries quietly push you here.
 - **Next.js** — a full framework on top of React: routing, server-side rendering, and server components out of the box. Reach for it when you are building a real product site and want SEO, fast first loads, and a backend in one place — not for a small internal SPA.
-

In one line: you already know how to make React work; understand closures, refs, the event loop, and typed state, lean on the right libraries, and test what the user sees — and now you understand *why* it works.

3 Common Mistakes to Avoid

These are the traps that bite when you use the new concepts from this guide. Quick  /  pairs — skim them now, come back when something breaks.

1. Reading a stale state value inside a closure

State is captured at render time. Inside one event handler, `count` does not change while your code runs — every read sees the same number. (Same idea as a C# lambda that captured a variable by value.)

 Three reads of the same stale value → adds 1, not 3.  Use the functional update `c ⇒ c + 1`; React hands you the latest value each time.

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  //  'count' is captured once this render, so all three see 0 → ends at 1.
  //   setCount(count + 1); setCount(count + 1); setCount(count + 1);

  //  React passes the freshest value into your updater each time → ends at 3.
  const addThree = () ⇒ {
    setCount(c ⇒ c + 1);
    setCount(c ⇒ c + 1);
    setCount(c ⇒ c + 1);
  };

  return <button onClick={addThree}>{count}</button>;
}
```

2. Using `useState` where you need `useRef` (and vice-versa)

Rule of thumb: **does the screen show this value?**

- Yes → `useState` (changing it should re-render).
- No, it just needs to survive across renders → `useRef` (changing it must *not* re-render).

Think of `useRef` as a private field on a class instance that lives across renders but never pokes the UI. A timer id is the classic case — storing it in `useState` triggers pointless re-renders.

 `const [timerId, setTimerId] = useState<number | null>(null)` — re-renders for a value nobody sees.  `const timerId = useRef<number | null>(null)` — silent storage; `seconds` stays in state

because the UI shows it.

```
import { useRef, useState } from 'react';

function Timer() {
  const [seconds, setSeconds] = useState(0);    // ✅ shown on screen → state
  const timerId = useRef<number | null>(null);  // ✅ hidden bookkeeping → ref

  const start = () => {
    if (timerId.current === null) return;
    timerId.current = window.setInterval(() => setSeconds(s => s + 1), 1000);
  };

  const stop = () => {
    if (timerId.current === null) {
      window.clearInterval(timerId.current);
      timerId.current = null;
    }
  };

  return (
    <div>
      <span>{seconds}s</span>
      <button onClick={start}>Start</button>
      <button onClick={stop}>Stop</button>
    </div>
  );
}
```

3. Forgetting the dependency array — or lying about it

The dependency array is a promise: "these are all the values this effect reads." Break the promise and the effect reads a stale value it never listed, or never re-runs when it should.

❌ `[]` means "run once" — so it never refetches when `userId` changes. ✅ List **every** reactive value the effect uses. (Bonus: the cleanup flag below also stops a slow first request from overwriting a newer one — a mini race-condition fix.)

```
import { useEffect, useState } from 'react';

function UserProfile({ userId }: { userId: string }) {
  const [name, setName] = useState('');

  // ❌ Lying with []: never re-runs when userId changes → shows the first user forever.
  //   useEffect(() => { fetchUser(userId).then(setName); }, []);

  // ✅ userId is listed, so the effect re-runs when it changes.
  useEffect(() => {
    let active = true;
    fetchUser(userId).then(n => { if (active) setName(n); });
    return () => { active = false; };
  }, [userId]);

  return <h1>{name}</h1>;
}

declare function fetchUser(id: string): Promise<string>;
```

4. `index` as a key, `==` instead of `===`, and trusting truthy/falsy

Index keys are fine only for a static list. The moment it can reorder or filter, React reuses the wrong DOM node — checkbox ticks and inputs "jump" to the wrong row. Use a stable id.

```
type Todo = { id: string; text: string };

function TodoList({ todos }: { todos: Todo[] }) {
  return (
    <ul>
      {/* ❌ index key breaks identity when the list reorders/filters:
        {todos.map((t, i) => <li key={i}>{t.text}</li>)} */}

      {/* ✅ stable id = stable identity */}
      {todos.map(t => <li key={t.id}>{t.text}</li>)}
    </ul>
  );
}
```

`==` vs `===`: `==` does surprise type-coercion (`0 == ''` is `true`). Always use `===` — it compares value *and* type, like a strict `.Equals` with no implicit conversion.

Truthy/falsy hides real values: `0` and `""` are falsy, so `if (count)` skips a valid `0`. Be explicit about what you actually mean.

```
// ❌ == coerces types: 0 == '' → true (surprise!)
// ✅ === checks value AND type, no surprises:
const a: number = 0;
console.log(a === 0); // true

// ❌ truthy check swallows a valid 0: if (count) { ... } skips when count === 0
// ✅ say what you mean:
const count = 0;
if (count > 0) {
  console.log('has items');
}
```

5. Mutating state or props directly

React decides whether to re-render by comparing references (`Object.is`). Push into the same array and the reference is unchanged, so React may skip the update. Props are read-only from the child — never write to them at all.

❌ `items.push(x); setItems(items)` — same reference, UI may not update. ✅ Build a new array/object, then set it. (Treat state like a C# `record` — you get a new copy, you don't edit in place.)

```
import { useState } from 'react';

function Cart() {
  const [items, setItems] = useState<string[]>([]);

  // ❌ Mutates the existing array - React sees the same reference:
  //   const add = (x: string) => { items.push(x); setItems(items); };

  // ✅ New array via spread → new reference → re-render.
  const add = (x: string) => setItems([ ...items, x]);

  return <button onClick={() => add('apple')}>{items.length}</button>;
}
```

6. Hand-rolling `fetch` in every component

Copy-pasting `useEffect` + `useState` + loading + error into each screen is a lot of code to get subtly wrong (missed error states, no caching, race conditions on fast typing). A data layer like **TanStack Query** does it once, correctly. Think of it as `MemoryCache` + a repository: caching, de-duping, and cancelling stale requests are built in.

❌ Manual fetch everywhere; forget the empty state; fast typing lets an old response overwrite a new one.

✅ One `useQuery` gives you `data`, `isLoading`, `isError`, caching, and per-key request handling.

Always render **all three** states — loading, error, and empty.

```
import { useQuery } from '@tanstack/react-query';

type User = { id: string; name: string };

function Users() {
  const { data, isLoading, isError } = useQuery({
    queryKey: ['users'],
    queryFn: (): Promise<User[]> => fetch('/api/users').then(r => r.json()),
  });

  if (isLoading) return <p>Loading ... </p>;
  if (isError) return <p>Something went wrong</p>;
  if (!data || data.length === 0) return <p>No users yet</p>; // ✅ empty state

  return <ul>{data.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

7. Reaching for Redux/Context for everything ("prop-drilling panic")

Passing a prop down two levels is normal — you don't need a global store for that. Match the tool to the need:

- One component's data → `useState`.
- A little shared UI state (theme, sidebar open) → Context or a small **Zustand** store.
- **Server data** → a query cache (TanStack Query), **not** global state.

❌ Everything in Redux, including fetched server data that then goes stale and needs manual syncing. ✅

Local state stays local; a tiny Zustand store handles genuine global UI; server data lives in the query cache.

```
import { create } from 'zustand';

// ✅ Real global UI state, no Redux boilerplate. (Server data still belongs in a query cache.)
type ThemeStore = {
  dark: boolean;
  toggle: () => void;
};

export const useTheme = create<ThemeStore>((set) => ({
  dark: false,
  toggle: () => set(s => ({ dark: !s.dark })),
}));
```

8. Sprinkling `any` / `as` to silence TypeScript

`as` is you *promising* the compiler a shape — it does zero runtime checking, so a wrong shape crashes later. `any` switches type-checking off entirely. Model the type instead.

❌ `const user = data as User` — a lie the compiler believes until it explodes. ✅ Use a **discriminated union** (like a `sealed` class hierarchy you `switch` / pattern-match on) so each state carries exactly the fields it has.

```
// ✅ Each variant is self-describing; TS narrows inside each case.
type Result =
  | { status: 'loading' }
  | { status: 'success'; user: { name: string } }
  | { status: 'error'; message: string };

function render(r: Result): string {
  switch (r.status) {
    case 'loading': return 'Loading ... ';
    case 'success': return r.user.name; // TS knows `user` exists here
    case 'error':   return r.message;   // and `message` here
  }
}
```

And when a value is genuinely unknown (JSON, a `catch` error), prefer `unknown` over `any` — it forces a check before use, like a safe cast gate.

```
// ❌ any turns off all checking: function parse(input: any) { return input.trim(); }
// ✅ unknown makes you prove the type first:
function parse(input: unknown): string {
  if (typeof input === 'string') return input.trim();
  return '';
}
```

9. Writing no tests — or testing the wrong thing

A test that pokes internal state breaks every time you refactor, even when the app still works. Test what the **user** sees and does. React Testing Library is like xUnit/NUnit plus a UI harness: you assert on visible output, not private fields.

❌ Assert `component.state.count === 1` (implementation detail; breaks on refactor). ✅ Click the button the user clicks, then assert the text the user reads.

```
import '@testing-library/jest-dom';
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { useState } from 'react';
import { expect, test } from 'vitest';

function Greeter() {
  const [hi, setHi] = useState(false);
  return (
    <div>
      <button onClick={() => setHi(true)}>Say hi</button>
      {hi && <p>Hi!</p>}
    </div>
  );
}

test('shows greeting after click', async () => {
  render(<Greeter />);
  await userEvent.click(screen.getByRole('button', { name: /say hi/i }));
  expect(screen.getByText('Hi!')).toBeInTheDocument();
});
```

10. Ignoring accessibility

A `<div onClick>` looks like a button but isn't one: no keyboard focus, no Enter/Space, no screen-reader announcement. And an input with no label is unusable for many people (and for your own tests using `getByLabelText`).

✗ `<div onClick={save}>Save</div>` and a bare `<input />` . ✓ Use the real `<button>` (focus + keyboard for free) and tie every `<label>` to its input via `htmlFor` / `id` .

```
// ✓ Real button = focus + Enter/Space + announced by screen readers.
function SaveButton({ onSave }: { onSave: () => void }) {
  return <button onClick={onSave}>Save</button>;
}

// ✓ label connected to input by matching htmlFor/id.
function EmailField() {
  return (
    <div>
      <label htmlFor="email">Email</label>
      <input id="email" type="email" />
    </div>
  );
}
```

11. Wrapping everything in `useMemo` / `useCallback` "for performance"

These aren't free — they add code, dependency arrays, and their own memory cost. Wrapping cheap work usually makes things *slower to read* and no faster to run. Reach for them only when a profiler shows a real cost, or when a memoized child needs a stable reference to skip re-renders.

✗ `useMemo(() => items.length, [items])` and `useCallback(() => {}, [])` on trivial work. ✓ Just compute the value. Optimise later, with a measurement in hand.

```
import { useState } from 'react';

function List() {
  const [items] = useState([1, 2, 3]);

  // ❌ Needless wrapping of trivial work:
  //   const total = useMemo(() => items.length, [items]);

  // ✅ Just compute it. Add memoization only when a profiler proves it's needed.
  const total = items.length;

  return <p>{total}</p>;
}
```

4 Concept Notes — Recall & Interview (copy by pen)

These are quick-recall cards for concept revision and interviews — one card per idea. Read the definition, glance at the pros/cons, and copy each by pen so it sticks.

JavaScript

Closures (and stale closures)

Definition:

- A closure is a function that remembers the variables from where it was created, even after that outer function has finished running.

✓ Advantages:

- Lets you keep private state and build helpers (counters, once-only functions, event handlers that remember data).

✗ Disadvantages / watch-outs:

- "Stale closure" — a function keeps an old captured value; common in React when a callback/effect closes over old state.

💡 Interview tip:

- Classic "What is a closure?": say "a function plus the variables it captured from its outer scope."

📄 Example:

- `const add = x => y => x + y;` — the inner arrow closes over `x`.

◆ C#/.NET:

- Same as a C# lambda capturing a local variable (`() => count`).

Lexical scope, hoisting & TDZ

Definition:

- Lexical scope = what a variable can see is fixed by where you write it; hoisting = declarations are lifted to the top of their scope; the TDZ (Temporal Dead Zone) is the gap where a `let` / `const` exists but can't be used yet.

✗ Disadvantages / watch-outs:

- `var` is function-scoped and hoisted as `undefined`; `let` / `const` are block-scoped and throw if touched inside the TDZ.

📄 Example:

- `console.log(x); let x = 1;` — throws (TDZ), not `undefined`.

◆ C#/.NET:

- The TDZ is like C# refusing to let you use a local before its declaration line.

`this` + `call` / `apply` / `bind`

Definition:

- `this` is the object a function runs against, and its value depends on how the function is called, not where it was written.

✓ Advantages:

- `call` / `apply` / `bind` let you set `this` manually and reuse one method on another object.

✗ Disadvantages / watch-outs:

- Regular functions get their own `this`; arrow functions don't — they use the surrounding `this` (ideal for callbacks).

📄 Example:

- `fn.call(obj, a, b)` vs `fn.apply(obj, [a, b])` vs `fn.bind(obj)` — `call/apply` run now; `bind` returns a new bound function.

◆ C#/.NET:

- Arrow-function `this` is like a captured `this` in a C# lambda; `bind` is like creating a delegate over a specific target.

Prototypes

Definition:

- Every JS object has a hidden link to a "prototype" object it inherits properties and methods from.

✓ Advantages:

- Shared methods live once on the prototype instead of being copied onto every instance (memory-efficient).

✗ Disadvantages / watch-outs:

- `class` in JS is just nicer syntax over prototypes — not a new C#-style type system.

📄 Example:

- `Object.getPrototypeOf(arr) ≡ Array.prototype` — arrays inherit `map` / `filter` from here.

◆ C#/.NET:

- Like a base class sharing members with derived instances, but resolved at runtime along a chain.

Event loop (micro vs macro tasks)

Definition:

- JS runs one thing at a time; finished async work is queued and the event loop runs it once the call stack is empty.

✓ **Advantages:**

- Non-blocking — the UI stays responsive while waiting on timers and network.

✗ **Disadvantages / watch-outs:**

- Microtasks (Promise callbacks, `queueMicrotask`) run before macrotasks (`setTimeout`, DOM events) — the ordering surprises people.

💡 **Interview tip:**

- Classic order question: sync code first → drain all microtasks → then the next macrotask; a resolved Promise `.then` beats `setTimeout(..., 0)`.

📄 **Example:**

- `setTimeout(f, 0); Promise.resolve().then(g);` — `g` runs before `f`.

◆ **C#/.NET:**

- Like how `await` schedules a continuation instead of blocking the thread.

== vs ===, coercion & truthy/falsy

Definition:

- `===` compares value and type with no conversion; `==` converts types first ("coercion") before comparing.

✓ **Advantages:**

- `===` is predictable and almost always the right choice.

✗ **Disadvantages / watch-outs:**

- Falsy values are `false, 0, "", null, undefined, NaN` — everything else is truthy; and `0 == ""` is `true`.

💡 **Interview tip:**

- Classic "`==` vs `===`": `==` coerces types before comparing, `===` doesn't — prefer `===`.

📄 **Example:**

- `null == undefined // true`, `null === undefined // false` — coercion in action.

◆ **C#/.NET:**

- `===` behaves like C# equality; JS `==` has no clean C# equivalent — avoid it.

Promise combinators (all / allSettled / race / any)

Definition:

- Helpers that run several promises together and combine their outcomes.

✓ Advantages:

- `all` = wait for all, fail fast; `allSettled` = wait for all, never rejects; `race` = first to settle (win or fail); `any` = first to succeed.

✗ Disadvantages / watch-outs:

- `all` rejects the moment any one rejects — use `allSettled` when you need every result regardless.

📄 Example:

- `await Promise.allSettled([a, b])` — returns an array of `{ status, value | reason }`.

◆ C#/.NET:

- `Promise.all` $\approx$ `Task.WhenAll`; `race` / `any` $\approx$ `Task.WhenAny`.

Key array & object methods

Definition:

- The everyday transforms: `map`, `filter`, `reduce`, `find`, `some`, `every`, `sort`, plus `Object.keys/values/entries`.

✓ Advantages:

- Declarative — you say what you want instead of writing manual loops, and you can chain them.

✗ Disadvantages / watch-outs:

- `sort` mutates the array and sorts as strings by default — pass a comparator and sort a copy.

📄 Example:

- `[...arr].sort((a, b)  $\Rightarrow$  a - b)` — numeric sort on a copy.

◆ C#/.NET:

- This is LINQ: `map` = `Select`, `filter` = `Where`, `reduce` = `Aggregate`, `find` = `FirstOrDefault`, `some` = `Any`, `every` = `All`.

Pure functions & immutability

Definition:

- A pure function returns the same output for the same input and changes nothing outside itself; immutability means making new copies instead of editing data in place.

✓ Advantages:

- Predictable and testable, and it lets React detect changes by reference — the basis of correct state updates.

✗ Disadvantages / watch-outs:

- Spread copies are shallow — nested objects still share references, so copy those levels too.

📄 Example:

- `const next = { ...state, done: true };` — new object, original untouched.

◆ C#/.NET:

- Like a `record` with `with` expressions, or returning new values instead of mutating fields.

Map & Set

Definition:

- `Map` is a key/value store that allows any key type and remembers insertion order; `Set` is a collection of unique values.

✓ Advantages:

- `Map` beats a plain object for frequent add/remove and non-string keys; `Set` dedupes instantly.

✗ Disadvantages / watch-outs:

- They don't serialize to JSON directly, and in React state you must replace them (not mutate) to trigger a render.

📄 Example:

- `[...new Set(arr)]` — a quick unique array.

◆ C#/.NET:

- `Map` $\approx$ `Dictionary<TKey, TValue>`, `Set` $\approx$ `HashSet<T>`.

Debounce & throttle

Definition:

- Debounce waits until activity stops before running; throttle runs at most once per time window.

✓ Advantages:

- Cut expensive work on rapid events — debounce for search inputs, throttle for scroll/resize.

✗ Disadvantages / watch-outs:

- In React, keep the timer in a `ref` (or use a library) so it survives re-renders.

📄 Example:

- `debounce(search, 300)` — fires 300ms after the last keystroke.

◆ C#/.NET:

- Like a `Timer` you reset on each event (debounce) or a cooldown gate (throttle).

React

useRef

Definition:

- A hook that holds a mutable value (`.current`) which survives re-renders but does not trigger a re-render when you change it.

✓ Advantages:

- Great for a DOM node reference or storing a timer / previous value without causing a render.

✗ Disadvantages / watch-outs:

- Changing `.current` won't update the UI — don't use it for data you display.

💡 Interview tip:

- Classic "useRef vs useState": state re-renders on change, ref doesn't — state for UI data, ref for behind-the-scenes values.

📄 Example:

- `const inputRef = useRef<HTMLInputElement>(null);` — then `inputRef.current?.focus()`.

◆ C#/.NET:

- Like a private field that persists across renders but has no change notification.

forwardRef / ref-as-prop (React 19)

Definition:

- Passing a `ref` down so a parent can reach a child's DOM node; in React 19 `ref` is a normal prop, so `forwardRef` is usually no longer needed.

✓ Advantages:

- Lets a parent focus, scroll, or measure a child input without extra state.

✗ Disadvantages / watch-outs:

- Older code still uses `forwardRef` — recognize both styles.

📄 Example:

- `function Input({ ref }: { ref?: Ref<HTMLInputElement> }) {}` — ref as a plain prop (React 19).

◆ C#/.NET:

- Like exposing a handle to a child control so the parent can call into it.

Fragments

Definition:

- A wrapper (`<...>`) that groups elements without adding an extra DOM node.

✓ Advantages:

- Return multiple elements without a useless wrapper `<div>`.

📄 Example:

- `<{a}{b}>` — use `<Fragment key={id}>` when you need a key inside a list.

Portals

Definition:

- Render children into a different DOM node (outside the parent) while keeping them in the same React tree.

✓ Advantages:

- Perfect for modals, tooltips, and toasts that must escape a parent's `overflow` / `z-index`.

✗ Disadvantages / watch-outs:

- Events still bubble through the React tree, not the DOM location — usually exactly what you want.

📄 Example:

- `createPortal(<Modal />, document.body)` — renders the modal at body level.

StrictMode

Definition:

- A dev-only wrapper that double-invokes some functions to help you catch impure code and missing effect cleanups.

✓ Advantages:

- Surfaces bugs early; it only double-runs in development, never in production.

✗ Disadvantages / watch-outs:

- Effects running twice in dev is expected — don't "fix" it by deleting StrictMode.

📄 Example:

- `<StrictMode><App /></StrictMode>` — wraps the root in dev.

◆ C#/.NET:

- Like extra assertions that only run in a Debug build, not Release.

useLayoutEffect vs useEffect

Definition:

- Both run after render; `useEffect` runs after the browser paints, while `useLayoutEffect` runs before paint, synchronously.

✓ Advantages:

- Use `useLayoutEffect` to measure the DOM and adjust it before the user sees a flicker.

✗ Disadvantages / watch-outs:

- `useLayoutEffect` blocks paint — use it rarely; `useEffect` is the default.

📄 Example:

- `useLayoutEffect(() => setWidth(ref.current!.offsetWidth), [])` — measure before paint.

useId

Definition:

- A hook that generates a stable, unique id that's safe across server and client rendering.

✓ Advantages:

- Ideal for linking a `<label htmlFor>` to an `<input id>` inside reusable components.

✗ Disadvantages / watch-outs:

- Don't use it as a list key — it's for element ids, not data identity.

📄 Example:

- `const id = useId();` — then `<label htmlFor={id}>`.

Suspense

Definition:

- A component that shows a fallback (like a spinner) while its children are loading — lazy code or `use()` data.

✓ Advantages:

- Declarative loading UI — no manual `isLoading` flag for lazy components.

📄 Example:

- `<Suspense fallback={<Spinner />}><LazyPage /></Suspense>` — spinner until ready.

◆ C#/.NET:

- Like an `await` boundary that shows a placeholder until the work completes.

useTransition & useDeferredValue

Definition:

- Tools to mark some state updates as low-priority so urgent updates (like typing) stay snappy.

✓ Advantages:

- `useTransition` wraps an update as non-urgent; `useDeferredValue` gives a lagging copy of a fast-changing value.

✗ Disadvantages / watch-outs:

- They don't make work faster — they just keep the UI responsive by deprioritizing it.

📄 Example:

- `const [isPending, startTransition] = useTransition();` — wrap heavy filtering inside `startTransition`.

React 19 Actions (useActionState / useFormStatus / useOptimistic / use())

Definition:

- A set of React 19 features that handle async form submissions, pending state, and optimistic UI with far less boilerplate.

✓ **Advantages:**

- `useActionState` runs an async action and tracks result + pending; `useFormStatus` reads the parent form's pending state; `useOptimistic` shows a temporary optimistic value; `use()` reads a promise or context during render.

✗ **Disadvantages / watch-outs:**

- `use()` follows Suspense rules — the component reading a promise must sit under a `<Suspense>` boundary.

📄 **Example:**

- `const [state, action, pending] = useActionState(submit, initial);` — wire `action` to `<form action={...}>`.

◆ **C#/.NET:**

- Actions ≈ an async command handler with built-in busy/result tracking, like an async MVVM command.

TanStack Query (useQuery / useMutation)

Definition:

- A data-fetching library that caches server data and handles loading, errors, refetching, and freshness for you.

✓ **Advantages:**

- `useQuery` for reads (cache + auto refetch); `useMutation` for writes (then invalidate the cache to refresh).

✗ **Disadvantages / watch-outs:**

- It's for server state, not client UI state — don't dump everything into it.

💡 **Interview tip:**

- Classic "why a data-fetching library?": it removes hand-written loading/error/caching/refetch code and dedupes requests.

📄 **Example:**

- `useQuery({ queryKey: ['todos'], queryFn: getTodos })` — cached fetch with status flags.

◆ **C#/.NET:**

- Like `MemoryCache` plus a repository layer with automatic cache invalidation.

Zustand & Redux Toolkit (when to use)

Definition:

- Two client-state managers: Zustand is a tiny hook-based store; Redux Toolkit is a structured, tooling-rich store.

✓ **Advantages:**

- Zustand = minimal boilerplate for small/medium shared state; Redux Toolkit = predictable, great DevTools, good for large apps and teams.

✗ **Disadvantages / watch-outs:**

- Don't reach for either if Context + useReducer (client state) or TanStack Query (server state) already covers it.

📄 **Example:**

- `const useStore = create<State>()(set => ({ count: 0, inc: () => set(s => ({ count: s.count + 1 }))) })))` — a Zustand store.

◆ **C#/.NET:**

- A global store ≈ a singleton service holding app state via DI.

Component composition (children) + HOC / render props (awareness)

Definition:

- Composition = build UIs by nesting components and passing `children`; HOCs (functions that wrap a component) and render props are older patterns for sharing logic.

✓ **Advantages:**

- `children` keeps components flexible and reusable; today custom hooks replace most HOC/render-prop use.

✗ **Disadvantages / watch-outs:**

- HOCs can create wrapper "hell" and prop-name clashes — prefer hooks now.

📄 **Example:**

- `<Card><Title /></Card>` where `Card` renders `{children}` — slot-style composition.

◆ **C#/.NET:**

- `children` is like a Blazor `RenderFragment` or WPF `ContentControl.Content`.

TypeScript with React

Utility types (Partial / Pick / Omit / Record / ReturnType)

Definition:

- Built-in generic types that transform an existing type instead of rewriting it by hand.

✓ **Advantages:**

- `Partial<T>` = all optional; `Pick<T, K>` = keep some keys; `Omit<T, K>` = drop some keys; `Record<K, V>` = build a map type; `ReturnType<F>` = grab a function's return type.

Example:

- `type Draft = Partial<User>;` — every field optional for an edit form.

◆ C#/.NET:

- Like generics/reflection helpers, but computed by the compiler at design time.

Discriminated unions for state

Definition:

- A union of object types sharing a literal "tag" field, so TypeScript can narrow to the exact shape you have.

✓ Advantages:

- Models async state precisely — no impossible combos like "loading, with data, and an error" all at once.

Example:

- `type S = { status: 'loading' } | { status: 'ok'; data: T } | { status: 'err'; error: string };` — then `switch` on `status`.

◆ C#/.NET:

- Like a sealed class hierarchy plus pattern matching (`switch` on the case).

Typing props / children / events / refs

Definition:

- The common React types: `ReactNode` for anything renderable, `PropsWithChildren<P>` to add `children`, `React.ChangeEvent<T>` for events, `Ref<T>` for refs.

✓ Advantages:

- Gives autocomplete and safety on props, event targets, and DOM handles.

Example:

- `(e: React.ChangeEvent<HTMLInputElement>) => e.target.value` — a typed input handler.

◆ C#/.NET:

- `ReactNode` ≈ accepting content/ `object`; typed events ≈ strongly-typed `EventArgs`.

unknown vs any + type guards

Definition:

- `any` turns type checking off; `unknown` is "some type" you must narrow before using; type guards are checks that prove the type.

✓ Advantages:

- `unknown` keeps safety for external data (API/JSON), and a guard narrows it to a real, usable type.

✗ Disadvantages / watch-outs:

- `any` silently spreads and hides bugs — reach for `unknown` plus a guard instead.

💡 Interview tip:

- Classic "unknown vs any": both accept anything, but `unknown` forces a check before use while `any` doesn't.

📄 Example:

- `if (typeof x === 'string') x.toUpperCase();` — the guard narrows `unknown`.

◆ C#/.NET:

- `any` ≈ `dynamic`; `unknown` ≈ `object` you must cast or pattern-match before use.

`as` / `non-null !`

Definition:

- `as` tells the compiler to treat a value as a given type; `!` asserts a value is not null/undefined.

✗ Disadvantages / watch-outs:

- Both are promises to the compiler with no runtime check — a wrong one causes a crash later.

📄 Example:

- `const el = document.getElementById('x')!;` — "trust me, not null" (use sparingly).

◆ C#/.NET:

- `as` ≈ a cast; `!` ≈ the null-forgiving `!` operator in C#.

`as const` & `satisfies`

Definition:

- `as const` freezes a value into its narrowest literal, readonly type; `satisfies` checks a value matches a type without widening it.

✓ Advantages:

- `as const` gives exact literal unions (great for config); `satisfies` validates while keeping the precise concrete type.

📄 Example:

- `const roles = ['admin', 'user'] as const;` — type is `readonly ['admin', 'user']`, not `string[]`.

◆ C#/.NET:

- `as const` $\approx$ `readonly` literal constants; `satisfies` $\approx$ compile-time interface conformance without losing the concrete type.

Ecosystem & Practices

Testing (Vitest + RTL + user-event)

Definition:

- Vitest runs the tests, React Testing Library renders components, and user-event simulates real user actions — then you assert on visible behavior.

✅ Advantages:

- Tests survive refactors because they check what the user sees, not internal implementation.

❌ Disadvantages / watch-outs:

- Querying by test-id or reaching into internal state is brittle — prefer role/label queries.

💡 Interview tip:

- Classic "behaviour vs implementation": assert on rendered output and interactions, not on internal functions or state.

📄 Example:

- `await user.click(screen.getByRole('button', { name: /save/i })))` — act like a user.

💎 C#/.NET:

- Vitest $\approx$ xUnit/NUnit; RTL + user-event $\approx$ a UI test harness driving the component.

Styling (CSS Modules / Tailwind / styled-components)

Definition:

- Three common approaches: CSS Modules (scoped `.module.css`), Tailwind (utility classes in markup), styled-components (CSS-in-JS).

✅ Advantages:

- CSS Modules = local scope, zero runtime; Tailwind = fast and consistent with no naming; styled-components = dynamic styles from props.

❌ Disadvantages / watch-outs:

- styled-components adds runtime cost; Tailwind markup looks noisy until you get used to it.

📄 Example:

- `import s from './Card.module.css'` then `className={s.card}` — a scoped class.

Accessibility (a11y) basics

Definition:

- Building UIs everyone can use — semantic HTML, labels, keyboard support, and ARIA only when needed.

✓ **Advantages:**

- Using real `<button>` / `<label>` / headings gives you keyboard and screen-reader support for free.

✗ **Disadvantages / watch-outs:**

- A clickable `<div>` isn't focusable or keyboard-usable — use a `<button>`, and add ARIA only to fill real gaps.

📄 **Example:**

- `<label htmlFor={id}>Name</label><input id={id} />` — label tied to its input.

Frameworks & rendering (Next.js; SPA vs SSR vs SSG vs RSC)

Definition:

- Where/when the HTML is built: SPA (in the browser), SSR (per request on the server), SSG (at build time), RSC (React Server Components render on the server and ship no JS); Next.js is the popular framework that offers all of these.

✓ **Advantages:**

- SSR/SSG improve first-load speed and SEO; RSC cut client JS; SPA is simplest for app-like dashboards.

✗ **Disadvantages / watch-outs:**

- The Vite apps you've built are SPAs — moving to SSR/RSC means adopting a framework like Next.js.

💡 **Interview tip:**

- Classic "SPA vs SSR": SPA renders in the browser (fast navigation, weaker SEO/first paint); SSR renders HTML on the server (better SEO/first paint, more server work).

📄 **Example:**

- `next build` → SSG pages; a page that fetches on the server → SSR/RSC.

◆ **C#/.NET:**

- SSR ≈ Razor Pages/MVC rendering HTML server-side; SPA ≈ a Blazor WASM / JS client app.